Agents & Real Chats

Day 3 — putting the pieces together.

You already built a server and a client. Today we make a real assistant. Plain English — read along.

What you've already built

Two days in. Here's the ground you're standing on.

Day 1 — a SERVER. A Python file with abilities: tools (get_weather, read_file), a resource (today's notes), a prompt (summarize).

Day 2 — a CLIENT. The code that runs the 4-step loop and lets an AI decide which tool to use.

Day 3 — today. We go from ONE tool call to a real, multi-step assistant that chats and remembers.

Quick recap: the 4-step loop

This is the engine from Day 2. Everything today sits on top of it.

1 · DISCOVER — the client asks the server: “what can you do?”

2 · DECIDE — the AI picks a tool for the question.

3 · EXECUTE — YOUR code runs the tool and gets a real result.

4 · SYNTHESIZE — the AI turns that result into a plain answer.

On Day 2 this ran ONCE and stopped. Today, the AI gets to run it again and again until the job is done.

Day 2 called ONE tool, then stopped

That's useful — but real questions often need several steps.

Your Day 2 client asked the AI, it picked one tool, you ran it, done. One and done.

Many questions need a chain. “What is France known for, and what's the weather in its capital?”

That needs two tools in a row: first look up France (find “Paris”), THEN get the weather for Paris.

The AI has to use the first result to decide the second step. One-and-done can't do that. We need a loop that keeps going.

What is an “agent”?

Three levels — you've already built the first two.

Level 1 — Chatbot. Reads your message, replies with text. (Plain ChatGPT.)

Level 2 — Tool use. Calls ONE tool, then replies. (Your Day 2 client.)

Level 3 — Agent. Calls tools in a row, and decides for itself when the task is done. (Today.)

An agent is just level 2, repeated. Call a tool, read the result, decide the next move, repeat — until it has the full answer.

Who runs the loop?

This is the whole difference between Day 2 and Day 3.

Day 2 — YOU ran it. Your code called one tool, then stopped. You were in charge.

An agent — the AI runs it. It calls a tool, reads the result, decides the next step, and stops when done.

Same tools. Same model. The only change is who controls the loop: you, or the AI.

A framework can run the loop for you

You wrote the loop by hand on Day 2. Now let a library do the boring part.

By hand: about 50 lines. Worth it — now you actually understand what happens.

With a framework: about 5 lines. Tools like smolagents run the exact same loop for you.

It's the SAME loop underneath. LangChain, LangGraph, n8n — all the same idea, just packaged differently.

Nothing new is happening. The framework hides the plumbing — the discover / decide / execute / synthesize loop is identical.

The three things a server offers (still true)

Today's chat uses all three at once. Remember who's in charge of each:

TOOLS → the AI pulls them. It decides when to call get_weather, read_file, run_sql…

RESOURCES → your app pushes them. You read the data (today's notes) and drop it into the chat as context.

PROMPTS → the user picks them. A person chooses a ready-made instruction, like “summarize this file.”

Only TOOLS go through the AI's decision loop. Resources and prompts are placed into the chat on purpose — by you, or by the user.

How a chat “remembers”

There's no magic memory. It's just a growing list of messages.

A chat is a list of turns. Each item is one message: system, user, assistant, or tool.

Every new turn is added to the end of the list.

The whole list is sent each time. So the AI can “see” everything said so far — that IS the memory.

```
messages = [
 {system: "you are helpful..."},
 {user: "what's the weather in TelAviv?"},
 {assistant: "call get_weather"}, {tool: "28C"},
 {user: "and what did I just ask?"}  <- it can see the earlier turns
]
```

One chat, MANY servers

The big idea for today — and how Claude Desktop and Goose really work.

A chat can connect to several servers at once. Your local one AND a cloud database, for example.

It merges all their tools into one list for the AI to choose from.

It remembers who owns each tool. When the AI picks one, the chat sends the call to the right server.

The AI has no idea the tools live in different places — that's the point of a shared standard.

Same AI — and can it even call tools?

Two things worth knowing before we run it.

It REASONS or it ACTS. Sometimes it answers from memory; sometimes it asks for a tool. Check tool_calls to tell which.

Not every model calls tools well. Small models fumble; bigger/newer ones are steadier. That's a model choice, not a bug.

We steer it with docstrings. A clear tool description tells the AI WHEN to use it. Docstrings are the steering wheel.

Real-world bumps (you'll hit these today)

Not mistakes in your thinking — this is what real tool integration feels like.

Changed the server code? Restart it. The server reads its code once, at startup.

Secrets go in a .env file — never in code. The APP owns keys and passwords, never the AI.

Small models sometimes reply oddly. They may write the tool call as plain text — real apps add a safety net.

Web tools have their own rules. A site may need a User-Agent, fresh certificates, or a clean search term. Normal friction.

Where this goes: the agent frameworks

You built the loop by hand. These libraries run it for you — same loop, different packaging.

smolagents — the light one we used. Small, simple, great for learning.

LangGraph — the loop as a typed graph with memory, retries, checkpoints. Popular for production.

CrewAI — give each agent a role and let several collaborate. The most-downloaded one in 2026.

OpenAI Agents SDK — the official path; easy if you already use the OpenAI API (you do).

LlamaIndex — same loop, but retrieval-first: answers grounded in your own documents (RAG).

Also out there: AutoGen (multi-agent), PydanticAI (type-safe), n8n (no-code, visual). Different names — same discover / decide / execute / synthesize loop you just built.

Enough theory.

Let's put it together.

Next: a chat that uses tools, remembers, and even talks to two servers at once.